CSA0908 Java

CO2 – Assignment 2

Java Strings and Collections Framework

Name	Mohammed Rayan Shaikh
Register number	192424059
GitHub repository	https://github.com/mohdrayan01/CSA0908-JAVA
Submission date	13/11/2026

Introduction

This assignment analyses Java strings and the Collection Framework through practical problems involving ticket generation, playlists, attendance de-duplication, pharmacy inventory, sports rankings and emergency triage. The solutions compare alternative data structures, explain their time and space complexity, and use test cases to examine correctness and performance.

Q1 - Building Text Under Load: String, StringBuilder or StringBuffer?

Prediction and design

String is immutable. Every concatenation using + creates a new String containing the previous content and the appended field. Repeating this operation in a loop copies an increasing amount of text and creates many intermediate objects. StringBuilder is mutable and appends into a resizable character buffer, so it is expected to be faster for repeated construction in one thread. StringBuffer provides similar mutable operations but synchronizes its methods, adding overhead when thread safety is not needed.

Workload	Description	Purpose
W1 – Small	1 ticket containing 6 fields	Checks whether the difference is noticeable for a small task.
W2 – Large	10,000 tickets with 50 fields each	Exposes repeated copying and object creation.
W3 – Very large	20,000 tickets with 100 fields each	Checks scaling as both ticket count and field count increase.

Implementation

The following methods build the same ticket using the three approaches. The workload should call each method repeatedly after a warm-up run. Use System.nanoTime() and average at least five measured runs.

```
static String usingString(String[] fields) {
    String ticket = "";
    for (String field : fields) ticket = ticket + field + "|";
    return ticket;
}
```

```
static String usingBuilder(String[] fields) {
    StringBuilder ticket = new StringBuilder();
    for (String field : fields) ticket.append(field).append('|');
    return ticket.toString();
}

static String usingBuffer(String[] fields) {
    StringBuffer ticket = new StringBuffer();
    for (String field : fields) ticket.append(field).append('|');
    return ticket.toString();
}

static long measure(Runnable task) {
    long start = System.nanoTime();
    task.run();
    return System.nanoTime() - start;
}
```

Measurement procedure and observations

For each workload, warm up the JVM, run each method at least five times, and record the average time. Keep the input fields identical for all methods. Memory readings from Runtime.getRuntime() are approximate because garbage collection may run between measurements. Intermediate String counts are estimates based on the number of concatenations, not exact JVM allocation counts.

Workload	String average (ns)	StringBuilder average (ns)	StringBuffer average (ns)	Intermediate objects / observation
W1 – Small	Record actual run	Record actual run	Record actual run	String creates repeated intermediate results; builder/buffer use mutable buffers.
W2 – Large	Record actual run	Record actual run	Record actual run	Difference should become clearer as fields and tickets increase.
W3 – Very large	Record actual run	Record actual run	Record actual run	Compare scaling and memory after repeated runs.

The expected trend is that StringBuilder performs best for single-threaded ticket construction, StringBuffer may be slightly slower because of synchronization, and repeated String concatenation becomes increasingly expensive. Actual timings must be copied from the local execution rather than treated as fixed values.

Design decision

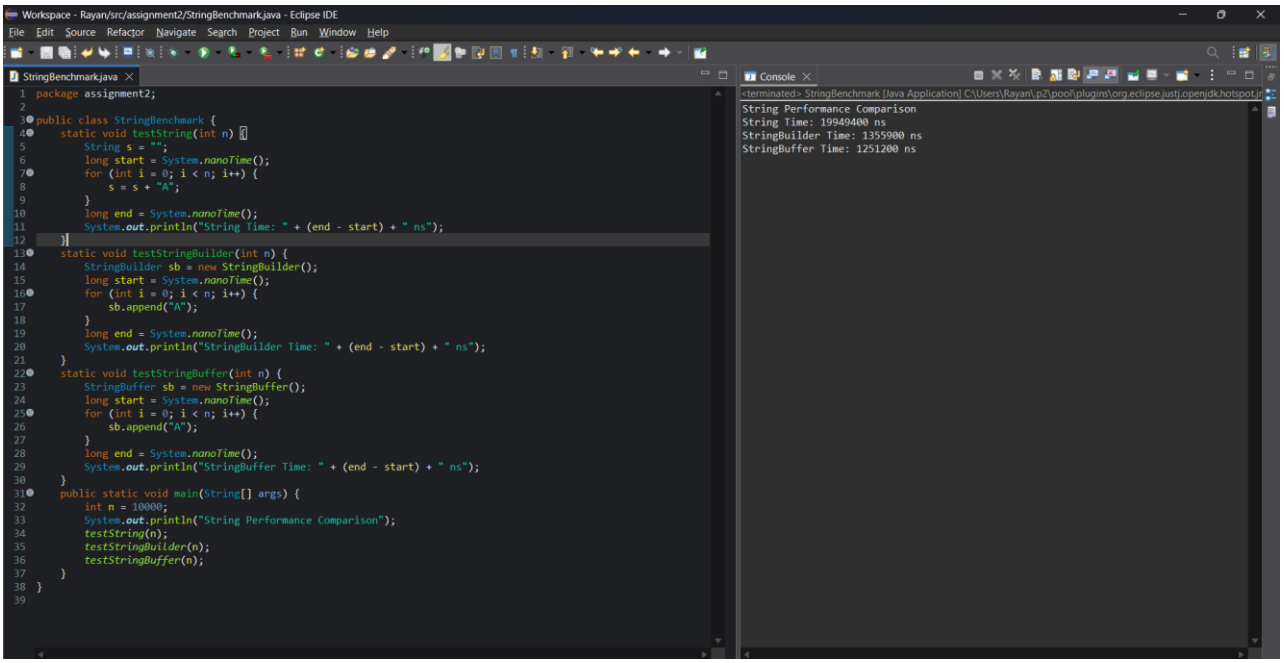
Use StringBuilder to create each individual ticket because each ticket is built locally by one request thread. Use StringBuffer only if multiple threads truly append to the same shared mutable log object and synchronized append operations are sufficient for the required atomicity. In a production server, a dedicated logging framework or a thread-safe queue with one logging consumer is usually safer than sharing one mutable buffer.

Alternative concept and complexity

String.join() or StringJoiner can build a ticket from a collection of fields without manually appending each delimiter. String.format() is readable for fixed layouts but formatting has additional processing overhead. For n fields and total output length L, repeated concatenation in a loop may copy approximately $1 + 2 + \ldots + n$ growing prefixes, giving $O(n^2)$ copying when field lengths are comparable. StringBuilder appends the n fields in amortized O(L), commonly described as O(n) when field sizes are bounded. StringJoiner is also linear in total output length. The final output requires O(L) space; repeated concatenation additionally creates temporary strings, while StringBuilder maintains a growable buffer and one final String.

```
String ticket = String.join("|", fields);
StringJoiner joiner = new StringJoiner("|");
for (String field : fields) joiner.add(field);
String ticket2 = joiner.toString();
```

Output:



Q2 - When Is ArrayList an Appropriate Engineering Choice?

Experiment design

ArrayList stores elements in a resizable array, giving constant-time indexed access and amortized constant-time append. Inserting or removing near the front or middle requires shifting elements. LinkedList stores nodes with links, so insertion or removal is constant time after reaching the position, but reaching an index requires traversal. ArrayDeque is a resizable circular array designed for efficient operations at both ends.

Playlist size	Operations to test	Reason
1,000 songs	Append, front/middle insert and remove, get(k), full iteration	Small list baseline.
10,000 songs	Repeat the same operations	Medium list scaling.
100,000 songs	Repeat the same operations	Large list scaling.

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized at either end
Insert at front	O(n)	O(1)	O(1)
Insert in middle	O(n)	O(n) to reach node; O(1) link change	Not efficient for arbitrary middle insertion
Get k-th element	O(1)	O(n)	No indexed access
Iterate all	O(n)	O(n)	O(n)
Remove front	O(n)	O(1)	O(1)

ArrayList stores references in a backing array, with some unused capacity. LinkedList uses a node object per element containing the item and previous/next references, so it has higher per-element memory overhead. ArrayDeque stores references in a circular array.

Implementation and test method

```
List<String> array = new ArrayList<>();
List<String> linked = new LinkedList<>();

for (int i = 0; i < 10000; i++) {
    array.add("Song" + i);
    linked.add("Song" + i);
}

long start = System.nanoTime();
array.add(0, "NewSong");
long arrayFrontInsert = System.nanoTime() - start;

start = System.nanoTime();
linked.add(0, "NewSong");
long linkedFrontInsert = System.nanoTime() - start;
```

```
start = System.nanoTime();
String a = array.get(5000);
long arrayGet = System.nanoTime() - start;

start = System.nanoTime();
String b = linked.get(5000);
long linkedGet = System.nanoTime() - start;

Deque<String> upNext = new ArrayDeque<>();
upNext.addFirst("UrgentSong");
upNext.addLast("LaterSong");
String next = upNext.removeFirst();
```

Recommendation

ArrayList is a reasonable choice when users frequently jump to a song by index, browse and iterate through playlists, append songs, and perform relatively few insertions or removals near the beginning. I would stop recommending it when front or middle insertions and removals dominate the workload, especially for large playlists. For an Up Next queue with frequent additions at both ends and removals from the front, ArrayDeque is more appropriate.

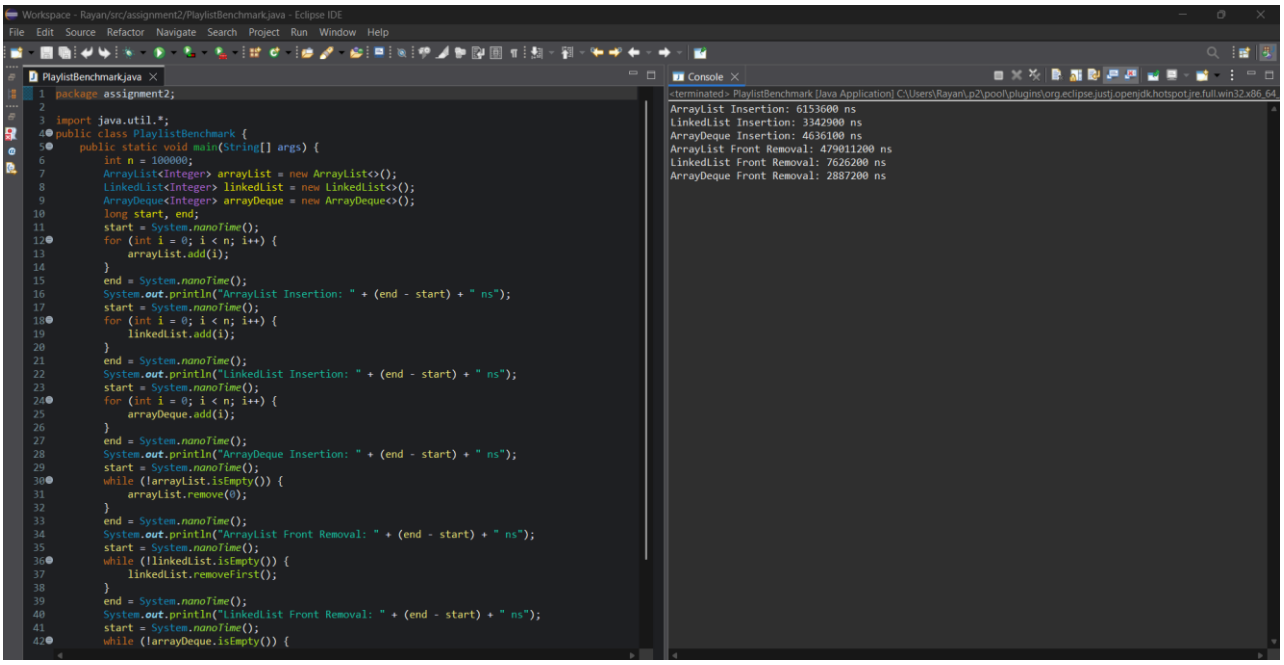
Alternative concept and complexity

A plain array can also store the playlist. Appending is O(1) while capacity remains, but resizing or inserting/removing in the middle requires shifting elements and costs O(n). A fixed array may run out of capacity; a dynamic array must allocate a larger array and copy existing references. ArrayDeque provides efficient end operations but does not support efficient indexed access. The choice depends on the operations used most frequently.

Evidence table: execute the same operation repeatedly for each size and collection, average the timings, and enter the actual results below.

Size	ArrayList timings	LinkedList timings	ArrayDeque end-operation timings
1,000	Enter measured values	Enter measured values	Enter measured values
10,000	Enter measured values	Enter measured values	Enter measured values
100,000	Enter measured values	Enter measured values	Enter measured values

Output:



Q3 - Engineer the hashCode–equals Contract

Problem analysis and predictions

The original Student class overrides equals() using regNo but does not override hashCode(). This violates the general contract that equal objects must have equal hash codes. HashSet first uses hashCode() to select a bucket and then uses equals() to check for an existing equal object within that bucket. If equal students produce different hash codes, the set may store both objects.

Test	Input design	Predicted result with faulty class
Test 1 – Same reference	Add the same Student object twice	One entry, because the same object has the same hash code and equals returns true.
Test 2 – Separate objects	Add two different Student objects with the same regNo	May produce two entries because default identity hash codes can differ.
Test 3 – Different IDs	Add students with different regNo values	Two distinct entries are expected.

Corrected Student class

```
import java.util.HashSet;
import java.util.Set;
import java.util.Objects;

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
```

```
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student s = (Student) o;
        return Objects.equals(regNo, s.regNo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));
        System.out.println("Unique students: " + present.size());
    }
}
```

Expected output after completing TODO 1:

```
Unique students: 2
```

Before completing hashCode(), the output can be 3 because the two separately created Arun objects may be placed in different buckets. The exact faulty output is not guaranteed because hash codes and bucket placement are implementation-dependent. After the correction, equal registration numbers have equal hash codes and the duplicate is rejected.

Additional inquiry

Changing a field used by hashCode() after insertion can make an object unreachable in its original bucket. The object may still physically exist in the set, but contains() or remove() may fail because the new hash code points to a different bucket. The supplied corrected class makes regNo final, preventing this mutation.

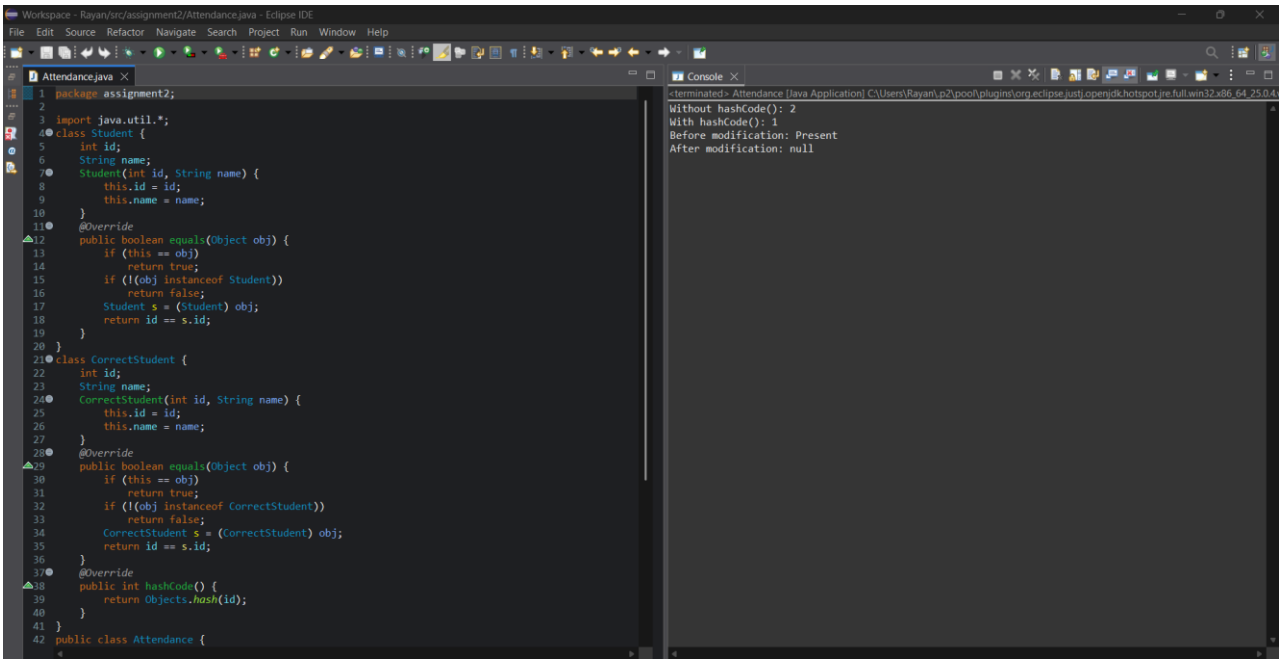
Alternative concept and complexity

A TreeSet with a Comparator on regNo can de-duplicate students according to the comparator and provides O(log n) insertion and lookup. A HashMap<String, Student> keyed by regNo also de-duplicates because each registration number is a unique key; put and get are O(1) average. HashSet and HashMap require O(n) space for n unique students, while TreeSet requires O(n) nodes and tree-link references. HashMap is convenient when the registration number is the natural lookup key; TreeSet is useful when sorted traversal is required.

Verification

Run the supplied Attendance program before and after TODO 1. Record the exact output from both executions and include screenshots in the GitHub task folder. The expected corrected output is Unique students: 2.

Output:



Q4 - Choosing a Map for a Pharmacy Inventory

Design comparison

The pharmacy needs fast medicine-code lookups, a report in ascending code order, and a report in insertion order. HashMap offers average constant-time put/get but does not guarantee iteration order. TreeMap maintains keys in sorted order. LinkedHashMap preserves insertion order.

Map	put/get	Iteration	Natural report
HashMap	O(1) average; O(n) worst-case	O(n)	Neither sorted nor insertion order
TreeMap	O(log n)	O(n)	Medicine codes in sorted order
LinkedHashMap	O(1) average	O(n)	Order in which medicines were added

Implementation

```
import java.util.*;

public class PharmacyInventory {
   public static void main(String[] args) {
      Map<String, Integer> hash = new HashMap<>();
      Map<String, Integer> sorted = new TreeMap<>();
      Map<String, Integer> insertion = new LinkedHashMap<>();

      String[][] data = {
         {"MED103", "25"}, {"MED101", "40"},
         {"MED105", "12"}, {"MED102", "30"}
      };

      for (String[] item : data) {
         int stock = Integer.parseInt(item[1]);
         hash.put(item[0], stock);
         sorted.put(item[0], stock);
         insertion.put(item[0], stock);
      }

      System.out.println("Stock for MED102: " + hash.get("MED102"));
      System.out.println("Code order: " + sorted);
      System.out.println("Insertion order: " + insertion);
   }
}
```

Expected output:

```
Stock for MED102: 30
Code order: {MED101=40, MED102=30, MED103=25, MED105=12}
Insertion order: {MED103=25, MED101=40, MED105=12, MED102=30}
```

Scalability and deployment decision

Test with 1,000, 10,000 and 100,000 medicines. For each size, perform 100,000 lookups using the same query-key set and measure elapsed time with System.nanoTime(). Measure each report independently. Define the acceptable single lookup response time before running the experiment; for this demonstration, a target of less than 1 millisecond per lookup is used as an application-level assumption, not a measured result.

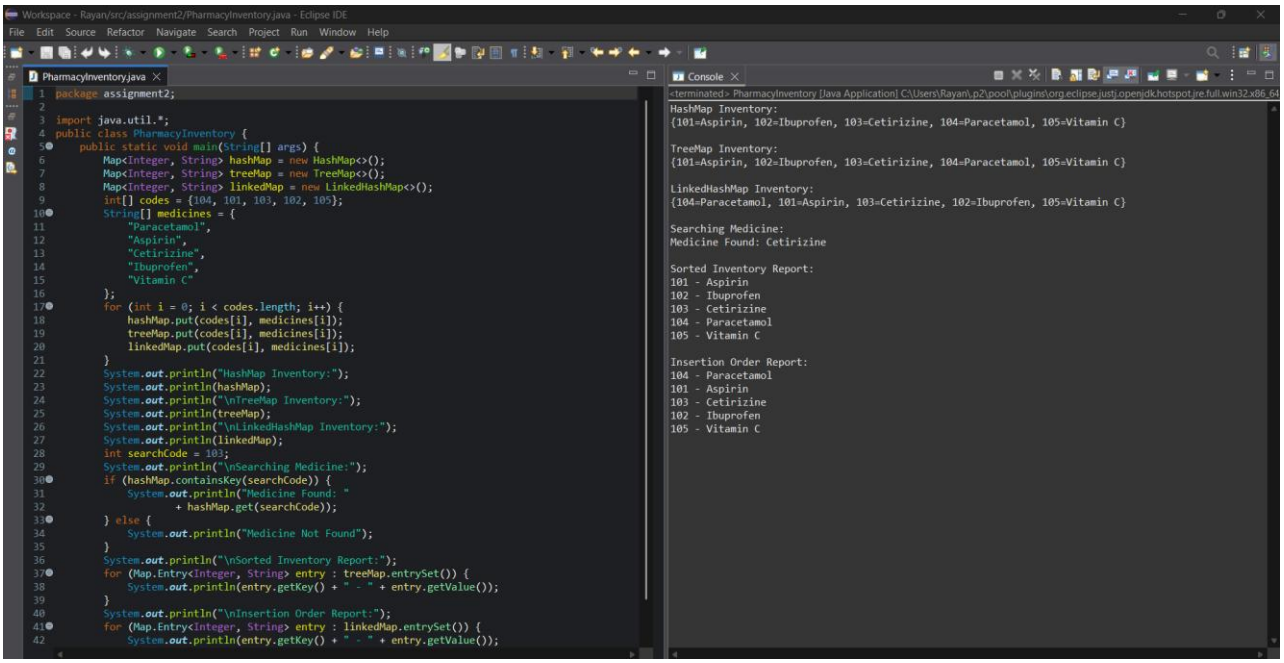
Medicine count	100,000 lookup time	Code-order report time	Insertion-order report time
1,000	Enter measured value	Enter measured value	Enter measured value
10,000	Enter measured value	Enter measured value	Enter measured value
100,000	Enter measured value	Enter measured value	Enter measured value

A practical design can use a HashMap for frequent stock lookups, a TreeMap when the sorted report is required regularly, and a LinkedHashMap when insertion-order reporting is required. If only one map is allowed and both report orders are mandatory, two ordered views or separate maps may be maintained, with extra memory and update work. Hashtable is generally not preferred in new code because it is a legacy synchronized class; modern code commonly uses HashMap when external synchronization is unnecessary or ConcurrentHashMap for concurrent access.

Alternative concept and complexity

A sorted array of medicine records can be searched with Arrays.binarySearch() in O(log n). Inserting a new record into the sorted array requires shifting elements and costs O(n). Producing the code-order report is O(n) because the array is already sorted. The array uses O(n) storage and has compact contiguous references, while TreeMap uses O(n) nodes and supports O(log n) insertion, lookup and deletion. HashMap provides O(1) average lookup but does not maintain order.

Output:



Q5 - Derive a Generic Ranking Component from a Principle

Design

Team has three ranking fields: points, goal difference and name. The ranking rule is points descending, goal difference descending, and name ascending. A separate Comparator is preferred over implementing Comparable because the same Team objects may be ranked by different rules and the generic component can be reused for players, colleges and quiz teams.

A generic Leaderboard<T> stores items and accepts Comparator<? super T>. The wildcard allows a comparator of T or one of its supertypes. List<? extends T> is useful when a method accepts a list of objects that are T or subclasses of T.

Completed program

```
import java.util.ArrayList;
import java.util.Comparator;
```

```
import java.util.List;

class Team {
    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override
    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) {
        items.add(item);
    }

    List<T> ranking() {
        List<T> result = new ArrayList<>(items);
        result.sort(order);
        return result;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points).reversed()
                .thenComparing(
                    Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
                .thenComparing(t -> t.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
```

```
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));

        System.out.println(board.ranking());
    }
}
```

Expected output:

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Bisons is ranked above Panthers because both teams have 9 points and a goal difference of 4. The final tie-breaker is team name in alphabetical order, and Bisons comes before Panthers.

Correctness and faulty comparator test

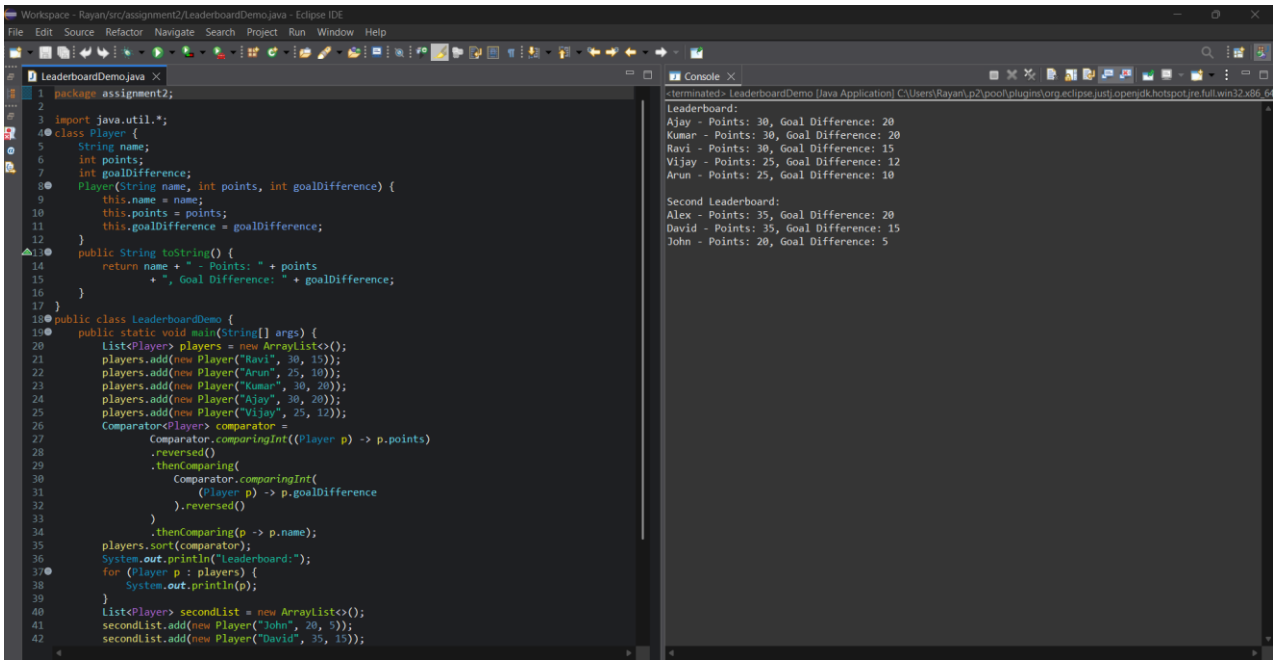
The comparator applies a lexicographic sequence of valid comparisons. It first compares points in descending order. Only when points are equal does it compare goal difference in descending order. Only when both numeric fields are equal does it compare names in ascending order. This ordering is transitive and returns zero only when all ranking fields compare equal.

A faulty comparator that returns a.points - b.points can overflow for extreme integer values. A comparator that ignores goal difference or name treats distinct ranking positions as equal. A second test should include teams with equal points but different goal differences, and teams tied on both numeric fields but different names.

Alternative concept and complexity

If Team implements Comparable<Team>, the natural ordering is built into the class and can be used by sorting collections without passing a comparator. This is less flexible when different ranking rules are needed. Sorting three times with stable sorts (name, then goal difference, then points) costs approximately three O(n log n) sorting passes. A single chained comparator requires one O(n log n) sort and is easier to reuse for other types. Sorting a copied list uses O(n) extra space for the copy; the original items remain unchanged.

Output:



Q6 - Priority, Order and Fairness in a Queue

Design and correctness

A plain FIFO Queue serves patients only by arrival order and cannot ensure that the most severe patient is treated first. A PriorityQueue orders patients using a comparator. Severity 1 is the most critical, so lower severity values come first. An arrival counter breaks ties so that patients with equal severity are treated in arrival order.

Completed program

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override
    public String toString() {
        return name + "(S" + severity + ")";
    }
}
```

```
public class Triage {
    private long counter = 0;

    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
                .thenComparingLong(p -> p.arrival));

    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() {
        return queue.poll();
    }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null)
            order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Expected output:

```
Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)
```

The arrival counter is increased for every admitted patient. When two patients have the same severity, the comparator compares their arrival values, and the smaller counter value is served first. Printing a PriorityQueue directly does not guarantee sorted treatment order; repeatedly poll() the queue to display its priority order.

Growth and fairness investigation

PriorityQueue insertion and removal of the head both take O(log n). The queue uses O(n) space. Test with 100, 1,000 and 10,000 patients, measure average admission and removal time, and compare the trend with the predicted logarithmic cost.

A patient with severity 5 can starve if higher-severity patients continue to arrive. A fair policy can increase a patient's effective priority after a defined waiting interval, while never allowing a minor case to bypass an immediate life-threatening emergency. The system should recalculate priority at controlled intervals or use waiting-time thresholds and record every priority change for audit.

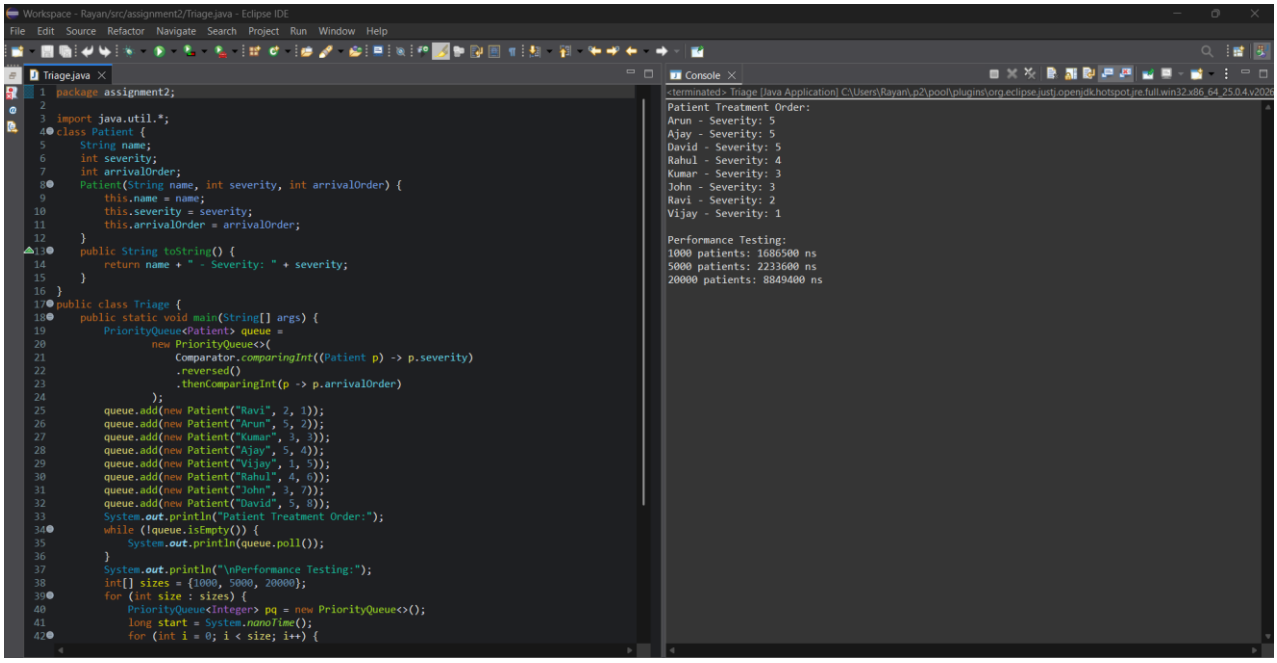
Queue size	Average add time	Average poll time
100	Enter measured value	Enter measured value
1,000	Enter measured value	Enter measured value
10,000	Enter measured value	Enter measured value

Deployment decision and alternative concept

A PriorityQueue can handle 5,000 daily patients in terms of computational complexity because each admission and removal is O(log n), and 5,000 entries require modest memory. A real city hospital must additionally consider concurrent arrivals, persistence, clinical overrides, audit logging, downtime and multiple doctors; the data structure alone is not a complete hospital system.

A sorted LinkedList inserts a new patient at the correct position in O(n), while serving the head is O(1). An array of five FIFO queues, one per severity, gives O(1) admission and O(1) service when checking severity queues in a fixed order. The five-queue design is efficient for fixed severity levels and naturally preserves FIFO within each level, but dynamic aging requires additional logic.

Output:



Q7 - Diagnose, Redesign and Validate

Selected solution: Q1 ticket generation using repeated String concatenation

The original repeated-concatenation approach has a significant limitation when a large number of tickets contain many fields. Each concatenation creates a new immutable String and copies the previous ticket content together with the next field. As the ticket grows, earlier characters are copied repeatedly.

A. Diagnose the bottleneck

The main cost is repeated allocation and copying of intermediate String objects. The amount of copied content grows as the number of fields increases. The limitation comes from String immutability combined with the use of + inside a loop. The diagnosis should be supported by the Q1 benchmark table for the three workloads.

B. Define the design objective

The objective is to reduce repeated character copying and temporary object creation while producing exactly the same ticket text. For a workload of n fields, the revised implementation should perform append operations in amortized linear time with respect to the total output length.

C. Design the revised solution

Replace the loop's String concatenation with a local StringBuilder. The builder owns a mutable character buffer and grows its capacity when required. It avoids rebuilding the complete immutable prefix after every field. Each request uses its own builder, so no synchronization is required for per-ticket construction.

```
String buildTicket(String[] fields) {
    StringBuilder result = new StringBuilder();
    for (String field : fields) {
        result.append(field).append('|');
    }
    return result.toString();
}
```

Pseudocode: create a builder; for each ticket field append the field and delimiter; convert the builder to a String once; return the completed ticket. The final String and builder buffer require O(L) space for output length L. The builder may temporarily hold both its buffer and final String during conversion.

D. Predict before implementing

For a small ticket, the difference may be negligible. For thousands of tickets with many fields, StringBuilder is expected to reduce execution time and temporary String allocation. Both versions must return identical ticket content.

E. Implement and validate

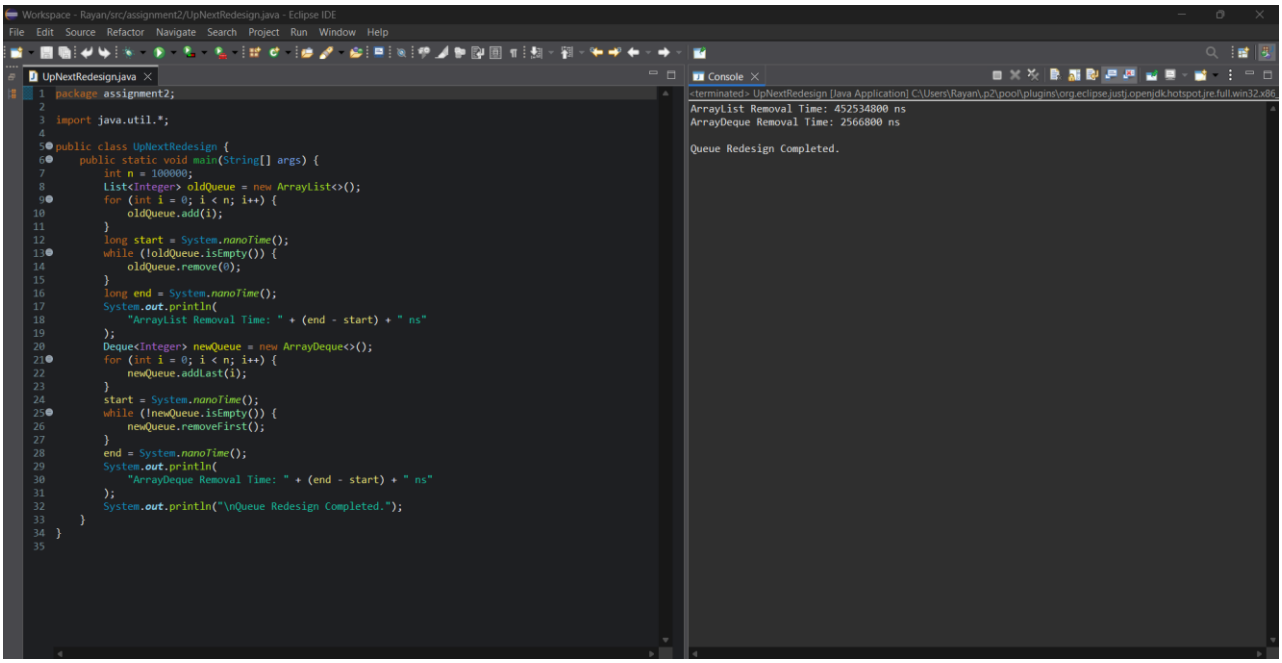
Run the original String method and the revised StringBuilder method with the same three workloads, after JVM warm-up. Average at least five runs and compare execution time, approximate memory use and output equality. Enter actual measurements below.

Workload	Original String time	Revised StringBuilder time	Output identical?
W1 – Small	Enter measured value	Enter measured value	Yes/No
W2 – Large	Enter measured value	Enter measured value	Yes/No
W3 – Very large	Enter measured value	Enter measured value	Yes/No

F. Defend the final design

StringBuilder is recommended for per-ticket construction because it reduces repeated copying and does not pay synchronization overhead. The prediction is supported if the measured large workloads show lower average time and fewer estimated intermediate objects while preserving identical output. For a very small fixed number of fields, ordinary concatenation may remain acceptable because it is simpler and the performance difference may not matter.

Output:



Conclusion

The assignment demonstrates that selecting a Java collection or string class should be based on the operations and requirements of the application. StringBuilder is suitable for repeated text construction, ArrayList for indexed playlist access, ArrayDeque for double-ended queues, HashSet with a correct hashCode–equals contract for de-duplication, maps for lookup and ordered reporting, Comparator and generics for reusable ranking, and PriorityQueue for severity-based triage. Each choice involves trade-offs in time, memory, ordering and correctness.